

Python Data Structures

List · Dictionary · Tuple · Set · All Methods & Examples

Complete Cheat Sheet · From Root to Advanced

OVERVIEW — 4 DATA STRUCTURES AT A GLANCE

Feature	List	Tuple	Set	Dict
Syntax	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}	{'k': 'v'}
Ordered	✓Yes	✓Yes	✗No	✓Yes (3.7+)
Mutable	✓Yes	✗No	✓Yes	✓Yes
Duplicates	✓Allowed	✓Allowed	✗Not allowed	✗Keys unique
Indexable	✓Yes [0]	✓Yes [0]	✗No	✓By key
Hashable	✗No	✓Yes	✗No	✗No
Use when	Ordered mutable collection	Fixed data / return values	Unique items / math ops	Key-value mapping

Mutable = can be changed after creation. Hashable = can be used as a dict key or in a set.

LIST

Ordered · Mutable · Allows Duplicates · Indexed

► Creating Lists

CREATING

```
empty    = []                # empty list
nums     = [1, 2, 3, 4, 5]
mixed    = [1, 'hello', 3.14, True] # any types
nested   = [[1,2], [3,4], [5,6]]  # list of lists
from_range = list(range(1, 6))    # [1,2,3,4,5]
from_str  = list('Python')        # ['P','y','t','h','o','n']
repeated = [0] * 5               # [0, 0, 0, 0, 0]
```

► Indexing & Slicing

INDEXING & SLICING

```
fruits = ['apple', 'banana', 'cherry', 'date', 'elderberry']
#       0         1         2         3         4
#      -5        -4        -3        -2        -1

print(fruits[0])    # apple      (first)
print(fruits[-1])   # elderberry (last)
```

```

print(fruits[1:3])    # ['banana', 'cherry'] (index 1,2)
print(fruits[:3])     # ['apple', 'banana', 'cherry'] (first 3)
print(fruits[2:])     # ['cherry', 'date', 'elderberry']
print(fruits[:2])     # ['apple', 'cherry', 'elderberry'] (every 2nd)
print(fruits[::-1])   # reversed list

```

```

OUT
apple
elderberry
['banana', 'cherry']
['apple', 'banana', 'cherry']
['cherry', 'date', 'elderberry']
['apple', 'cherry', 'elderberry']
['elderberry', 'date', 'cherry', 'banana', 'apple']

```

► All List Methods

Method / Syntax	What It Does & Example
<code>lst.append(x)</code>	Add x to END of list → <code>[1,2].append(3) → [1,2,3]</code>
<code>lst.insert(i, x)</code>	Insert x at index i → <code>[1,3].insert(1,2) → [1,2,3]</code>
<code>lst.extend(iterable)</code>	Add all items from iterable → <code>[1].extend([2,3]) → [1,2,3]</code>
<code>lst.remove(x)</code>	Remove FIRST occurrence of x → <code>ValueError</code> if not found
<code>lst.pop()</code>	Remove & return LAST item → <code>[1,2,3].pop() → 3</code>
<code>lst.pop(i)</code>	Remove & return item at index i → <code>[1,2,3].pop(0) → 1</code>
<code>lst.clear()</code>	Remove ALL items → <code>[]</code> (empty list)
<code>lst.index(x)</code>	Return index of first x → <code>ValueError</code> if not found
<code>lst.count(x)</code>	Count occurrences of x → <code>[1,2,1].count(1) → 2</code>
<code>lst.sort()</code>	Sort IN-PLACE ascending → modifies original list
<code>lst.sort(reverse=True)</code>	Sort IN-PLACE descending
<code>lst.reverse()</code>	Reverse IN-PLACE → <code>[1,2,3] → [3,2,1]</code>
<code>lst.copy()</code>	Shallow copy → new list, same object references
<code>sorted(lst)</code>	Returns NEW sorted list → original unchanged
<code>reversed(lst)</code>	Returns iterator in reverse → <code>list(reversed(lst))</code>
<code>len(lst)</code>	Number of items → <code>len([1,2,3]) → 3</code>
<code>max(lst)</code>	Largest item → <code>max([3,1,2]) → 3</code>
<code>min(lst)</code>	Smallest item → <code>min([3,1,2]) → 1</code>
<code>sum(lst)</code>	Sum of all items → <code>sum([1,2,3]) → 6</code>
<code>x in lst</code>	Membership check → <code>3 in [1,2,3] → True</code>
<code>lst1 + lst2</code>	Concatenate two lists → <code>[1,2]+[3,4] → [1,2,3,4]</code>
<code>lst * n</code>	Repeat list → <code>[0]*3 → [0,0,0]</code>

► List Methods — Live Examples

EXAMPLE

```

nums = [3, 1, 4, 1, 5, 9, 2, 6]

# Adding items
nums.append(7)          # [3,1,4,1,5,9,2,6,7]
nums.insert(0, 0)       # [0,3,1,4,1,5,9,2,6,7]
nums.extend([8, 10])    # adds 8 and 10 at end

# Removing items
nums.remove(1)          # removes FIRST 1
popped = nums.pop()     # removes & returns last item
popped_idx = nums.pop(0) # removes & returns index 0

# Searching
print(nums.index(5))     # index of 5
print(nums.count(1))     # how many 1s

# Sorting
nums.sort()             # in-place ascending
nums.sort(reverse=True) # in-place descending
new_sorted = sorted(nums) # returns new sorted list

# Copy
copy1 = nums.copy()     # shallow copy
copy2 = nums[:]         # also a copy (slice trick)

```

► Sorting with key= (Advanced)

SORTING WITH key=

```

words = ['banana', 'fig', 'apple', 'elderberry', 'date']

# Sort by length
print(sorted(words, key=len))

# Sort alphabetically, case-insensitive
print(sorted(words, key=str.lower))

# Sort list of dicts by a field
students = [
    {'name': 'Priya', 'score': 88},
    {'name': 'Aman', 'score': 95},
    {'name': 'Rohit', 'score': 72},
]
ranked = sorted(students, key=lambda s: s['score'], reverse=True)
for s in ranked:
    print(f"{s['name']}: {s['score']}")

```

```

OUT      ['fig', 'date', 'apple', 'banana', 'elderberry']
         ['apple', 'banana', 'date', 'elderberry', 'fig']
         Aman: 95
         Priya: 88
         Rohit: 72

```

► List Unpacking

UNPACKING

```

a, b, c = [10, 20, 30]    # basic unpack
print(a, b, c)            # 10 20 30

```

```
first, *rest = [1,2,3,4,5]    # starred unpack
print(first)    # 1
print(rest)     # [2,3,4,5]

*start, last = [1,2,3,4,5]
print(start)    # [1,2,3,4]
print(last)     # 5

a, *mid, z = [1,2,3,4,5]
print(a, mid, z)    # 1 [2,3,4] 5
```

OUT

```
10 20 30
1
[2,3,4,5]
[1,2,3,4]
5
1 [2, 3, 4] 5
```

📖 DICTIONARY

Key-Value Pairs · Ordered (3.7+) · Mutable · Keys Unique

► Creating Dictionaries

CREATING

```
empty    = {}                                # empty dict
person   = {'name': 'Nishant', 'age': 25}
mixed    = {1: 'one', 'two': 2, (3,): 'three'} # various key types

# dict() constructor
d1 = dict(name='Aman', city='Delhi')          # keyword args
d2 = dict([('a',1), ('b',2)])                  # from list of tuples

# dict.fromkeys() – create dict with default values
keys    = ['x', 'y', 'z']
zeroed  = dict.fromkeys(keys, 0)               # {'x':0, 'y':0, 'z':0}

# Nested dict
students = {
    'Aman': {'age': 22, 'grade': 'A'},
    'Priya': {'age': 21, 'grade': 'B+'},
}
```

► Accessing & Modifying

ACCESS & MODIFY

```
d = {'name': 'Nishant', 'age': 25, 'city': 'Delhi'}

# Access
print(d['name'])          # Nishant
print(d.get('age'))       # 25
print(d.get('phone', 'N/A')) # N/A – default if key missing

# Add / Update
d['email'] = 'n@gmail.com' # add new key
d['age']   = 26             # update existing

# Update multiple keys at once
d.update({'age': 27, 'score': 95})

# Delete
del d['city']              # KeyError if not found
val = d.pop('email')       # remove & return value
d.popitem()               # remove & return LAST inserted pair
```

► All Dictionary Methods

Method / Syntax	What It Does & Example
<code>d.get(key, default)</code>	Safe access — returns default if key missing (no error)

<code>d.keys()</code>	Returns view of all keys → <code>dict_keys(['a','b'])</code>
<code>d.values()</code>	Returns view of all values → <code>dict_values([1,2])</code>
<code>d.items()</code>	Returns view of (key,value) tuples
<code>d.update(other)</code>	Merge another dict into d → adds/overwrites keys
<code>d.pop(key)</code>	Remove key & return its value → <code>KeyError</code> if missing
<code>d.pop(key, default)</code>	Remove key & return default if missing → safe
<code>d.popitem()</code>	Remove & return last inserted (key,val) pair
<code>d.setdefault(k, v)</code>	Return value if key exists; else insert k:v and return v
<code>d.copy()</code>	Shallow copy of the dict
<code>d.clear()</code>	Remove all items → empty dict <code>{}</code>
<code>dict.fromkeys(keys, v)</code>	Class method — create dict from list of keys with value v
<code>key in d</code>	Check if key exists → <code>'name' in d</code> → <code>True</code>
<code>len(d)</code>	Number of key-value pairs
<code>del d[key]</code>	Delete key → <code>KeyError</code> if not found

► Dictionary Methods — Live Examples

EXAMPLE

```
inventory = {'apple': 50, 'banana': 30, 'cherry': 20}

# Safe access
print(inventory.get('mango', 0))      # 0 — no KeyError

# setdefault — get or initialize
inventory.setdefault('date', 0)      # adds date:0 if missing
inventory['date'] += 10               # now date:10

# Merging dicts
new_stock = {'elderberry': 15, 'apple': 60}
inventory.update(new_stock)           # apple updated to 60

# Iterate
for item, qty in inventory.items():
    print(f'{item:<12} : {qty} units')

# Sort by value
sorted_inv = dict(sorted(inventory.items(), key=lambda x: x[1], reverse=True))
print(sorted_inv)
```

► Dictionary Comprehension

DICT COMPREHENSION

```
# Square of numbers
squares = {x: x**2 for x in range(1,6)}
print(squares)  # {1:1, 2:4, 3:9, 4:16, 5:25}

# Filter — only items with value > 2
big = {k:v for k,v in squares.items() if v > 4}
```

```

print(big)          # {3:9, 4:16, 5:25}

# Flip keys and values
flipped = {v:k for k,v in squares.items()}
print(flipped)      # {1:1, 4:2, 9:3, 16:4, 25:5}

# Word frequency counter
words = ['apple','banana','apple','cherry','banana','apple']
freq = {w: words.count(w) for w in set(words)}
print(freq)         # {'cherry':1, 'banana':2, 'apple':3}

```

```

OUT      {1:1, 2:4, 3:9, 4:16, 5:25}
         {3:9, 4:16, 5:25}
         {1:1, 4:2, 9:3, 16:4, 25:5}
         {'cherry':1, 'banana':2, 'apple':3}

```

► Nested Dict Access & defaultdict

ADVANCED

```

# Nested dict
school = {
    'Aman': {'math': 90, 'science': 85},
    'Priya': {'math': 88, 'science': 92},
}
print(school['Priya']['science'])  # 92

# defaultdict – auto-creates missing keys
from collections import defaultdict

word_len = defaultdict(list)      # default value is empty list
for word in ['cat','dog','ant','bear','crow']:
    word_len[len(word)].append(word)

print(dict(word_len))
# {3: ['cat','dog','ant'], 4: ['bear','crow']}

```

```

OUT      92
         {3: ['cat', 'dog', 'ant'], 4: ['bear', 'crow']}

```

TUPLE

Ordered · Immutable · Allows Duplicates · Hashable

► Creating Tuples

CREATING

```
empty    = ()                # empty tuple
single   = (42,)             # MUST have trailing comma!
coords   = (10.5, 20.3)      # two values
mixed    = (1, 'hello', 3.14, True)
nested   = ((1,2), (3,4), (5,6))
from_lst = tuple([1,2,3])    # convert list → tuple
from_str = tuple('abc')      # ('a','b','c')

# Packing (no parentheses needed)
point = 3, 7                 # same as (3, 7)
print(type(point))           # <class 'tuple'>
```

🔗 single = (42) is just an integer! You MUST write (42,) with a trailing comma to make it a tuple.

► Indexing, Slicing, Unpacking

INDEXING & UNPACKING

```
rgb = (255, 128, 0)

print(rgb[0])    # 255
print(rgb[-1])   # 0
print(rgb[1:])   # (128, 0)

# Unpacking
r, g, b = rgb
print(r, g, b)   # 255 128 0

# Swap variables using tuple
a, b = 10, 20
a, b = b, a      # pythonic swap
print(a, b)     # 20 10

# Extended unpacking
first, *rest = (1,2,3,4,5)
print(first, rest) # 1 [2,3,4,5]
```

OUT

```
255
0
(128, 0)
255 128 0
20 10
1 [2, 3, 4, 5]
```

► Tuple Methods (only 2!)

Method / Syntax

What It Does & Example

<code>t.count(x)</code>	Count occurrences of x → <code>(1,2,1,3).count(1)</code> → 2
<code>t.index(x)</code>	Return index of first x → <code>(1,2,3).index(2)</code> → 1
<code>len(t)</code>	Number of items → <code>len((1,2,3))</code> → 3
<code>x in t</code>	Membership check → <code>2 in (1,2,3)</code> → True
<code>t1 + t2</code>	Concatenate → <code>(1,2)+(3,4)</code> → <code>(1,2,3,4)</code>
<code>t * n</code>	Repeat → <code>(0,1)*3</code> → <code>(0,1,0,1,0,1)</code>
<code>max(t)</code>	Largest item
<code>min(t)</code>	Smallest item
<code>sorted(t)</code>	Returns new SORTED LIST (not tuple)
<code>tuple(lst)</code>	Convert list/iterable to tuple

🔗 Tuples have only 2 methods (count & index) because they're immutable. But they support all sequence operations like indexing, slicing, len, in, +, *.

► Why Use Tuples? — Real Use Cases

REAL USE CASES

```
# 1. Return multiple values from a function
def min_max(nums):
    return min(nums), max(nums) # returns a tuple
lo, hi = min_max([3,1,9,5])
print(lo, hi)                  # 1 9

# 2. Named tuples – self-documenting
from collections import namedtuple
Point = namedtuple('Point', ['x','y'])
Color = namedtuple('Color', ['r','g','b'])

p = Point(10, 20)
c = Color(255, 128, 0)
print(p.x, p.y)               # 10 20
print(c.r, c.g, c.b)          # 255 128 0

# 3. Dict keys (tuples are hashable, lists are not)
locations = {(28.6, 77.2): 'Delhi', (19.0, 72.8): 'Mumbai'}
print(locations[(28.6, 77.2)]) # Delhi

# 4. Immutability = data safety (config, constants)
DB_CONFIG = ('localhost', 5432, 'mydb') # can't accidentally change
```

OUT

```
1 9
10 20
255 128 0
Delhi
```

SET

Unordered · Mutable · No Duplicates · Not Indexed

► Creating Sets

CREATING

```
empty = set()                # NOT {} – that creates an empty dict!
nums = {1, 2, 3, 4, 5}
dedup = {1, 2, 2, 3, 3, 3}   # {1, 2, 3} – duplicates removed
from_lst = set([1,2,2,3,4,4]) # convert list (and deduplicate)
from_str = set('hello')      # {'h','e','l','o'} – unique chars

print(from_lst)   # {1, 2, 3, 4}
print(from_str)   # {'h', 'e', 'l', 'o'} (order not guaranteed)
```

❗ `empty = {}` creates an empty DICT, not a set. Always use `set()` for empty sets.

► All Set Methods

Method / Syntax	What It Does & Example
<code>s.add(x)</code>	Add x to set → <code>{1,2}.add(3) → {1,2,3}</code>
<code>s.remove(x)</code>	Remove x → <code>KeyError</code> if x not in set
<code>s.discard(x)</code>	Remove x → NO error if x not in set (safe)
<code>s.pop()</code>	Remove & return ARBITRARY item (order undefined)
<code>s.clear()</code>	Remove all items → empty set <code>set()</code>
<code>s.copy()</code>	Shallow copy of the set
<code>s.update(other)</code>	Add all items from other into s
<code>s.union(t)</code>	Items in s OR t → <code>s t</code>
<code>s.intersection(t)</code>	Items in BOTH s AND t → <code>s & t</code>
<code>s.difference(t)</code>	Items in s but NOT in t → <code>s - t</code>
<code>s.symmetric_difference(t)</code>	Items in s OR t but NOT both → <code>s ^ t</code>
<code>s.issubset(t)</code>	True if every item of s is in t → <code>s <= t</code>
<code>s.issuperset(t)</code>	True if s contains every item of t → <code>s >= t</code>
<code>s.isdisjoint(t)</code>	True if s and t have NO items in common
<code>s.intersection_update(t)</code>	Keep only items in BOTH s AND t (in-place)
<code>s.difference_update(t)</code>	Remove from s all items found in t (in-place)
<code>x in s</code>	Membership check — O(1) speed! → <code>3 in {1,2,3} → True</code>
<code>len(s)</code>	Number of items
<code>frozenset(s)</code>	Immutable version of a set (hashable)

► Set Operations — Math Style

SET MATH

```
a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7, 8}

print(a | b)      # Union      → {1,2,3,4,5,6,7,8}
print(a & b)      # Intersection → {4,5}
print(a - b)      # Difference → {1,2,3}   (in a, not in b)
print(b - a)      # Difference → {6,7,8}   (in b, not in a)
print(a ^ b)      # Symmetric  → {1,2,3,6,7,8} (not in both)
```

OUT	{1, 2, 3, 4, 5, 6, 7, 8}
	{4, 5}
	{1, 2, 3}
	{6, 7, 8}
	{1, 2, 3, 6, 7, 8}

► Set Methods — Live Examples

EXAMPLE

```
s = {10, 20, 30}

s.add(40)          # {10,20,30,40}
s.discard(99)      # no error even though 99 not in set
s.remove(20)       # {10,30,40}

# Subset / Superset
print({1,2}.issubset({1,2,3,4}))    # True
print({1,2,3,4}.issuperset({2,3}))  # True
print({1,2}.isdisjoint({3,4}))      # True — no common items

# Practical: Remove duplicates from a list (preserve order in 3.7+)
data = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
unique = list(dict.fromkeys(data))    # preserves order
unique2 = list(set(data))             # may not preserve order
print(unique)                        # [3, 1, 4, 5, 9, 2, 6]

# Fast membership check — O(1) vs list O(n)
large_set = set(range(1_000_000))
print(999_999 in large_set)          # True — instant!
```

OUT	True
	True
	True
	[3, 1, 4, 5, 9, 2, 6]
	True

🔍 Set membership check 'x in set' is O(1) — instant regardless of size. List is O(n) — slower as list grows. Always use sets for large membership checks.

► frozenset — Immutable Set

FROZENSET

```
fs = frozenset([1, 2, 3, 4])
print(fs)                # frozenset({1,2,3,4})
```

```
# fs.add(5)                # AttributeError – immutable!

# frozenset can be used as dict key or inside another set
graph = {
    frozenset({1,2}): 'edge A',
    frozenset({2,3}): 'edge B',
}
print(graph[frozenset({1,2})])  # 'edge A'
```

OUT

```
frozenset({1, 2, 3, 4})
edge A
```

COMMON PATTERNS & REAL-WORLD IDIOMS

► Shallow Copy vs Deep Copy

COPY BEHAVIOUR

```
import copy

original = [[1,2,3],[4,5,6]]

# Shallow copy – copies outer list, inner lists still SHARED
shallow = original.copy()      # or original[:]
shallow[0].append(99)          # modifies original[0] too!
print(original[0])             # [1,2,3,99] ← side effect!

# Deep copy – fully independent copy
original2 = [[1,2,3],[4,5,6]]
deep = copy.deepcopy(original2)
deep[0].append(99)             # ONLY changes deep
print(original2[0])            # [1,2,3] ← safe!
```

```
OUT      [1, 2, 3, 99] ← Shallow: side effect!
         [1, 2, 3]  ← Deep: safe!
```

► Counting Items — Counter

COUNTER

```
from collections import Counter

words = ['apple','banana','apple','cherry','banana','apple']
c = Counter(words)
print(c)                # Counter({'apple':3,'banana':2,'cherry':1})
print(c.most_common(2)) # [('apple',3),('banana',2)]

# Count characters in a string
chars = Counter('mississippi')
print(chars) # Counter({'s':4,'i':4,'p':2,'m':1})
```

```
OUT      Counter({'apple': 3, 'banana': 2, 'cherry': 1})
         [('apple', 3), ('banana', 2)]
         Counter({'s': 4, 'i': 4, 'p': 2, 'm': 1})
```

► Combining Data Structures

LIST OF DICTS PATTERN

```
# List of dicts – most common real-world pattern
employees = [
    {'name': 'Aman', 'dept': 'Tech', 'salary': 80000},
    {'name': 'Priya', 'dept': 'Design', 'salary': 75000},
    {'name': 'Rohit', 'dept': 'Tech', 'salary': 90000},
    {'name': 'Neha', 'dept': 'Design', 'salary': 70000},
]

# Filter Tech employees
tech = [e for e in employees if e['dept'] == 'Tech']
```

```
# Average salary per dept
from collections import defaultdict
dept_salaries = defaultdict(list)
for emp in employees:
    dept_salaries[emp['dept']].append(emp['salary'])

for dept, sals in dept_salaries.items():
    print(f'{dept}: avg Rs.{sum(sals)//len(sals)}')
```

OUT

Tech: avg Rs.85000
Design: avg Rs.72500

🔍 QUICK REFERENCE — ALL METHODS AT A GLANCE

LIST METHODS

Method	Action	Returns
append(x)	Add to end	None (in-place)
insert(i, x)	Insert at index	None (in-place)
extend(iter)	Add all from iterable	None (in-place)
remove(x)	Remove first x	None / KeyError
pop(i)	Remove at index i	Removed item
clear()	Remove all	None
index(x)	Find first x	Index int
count(x)	Count x	int
sort()	Sort in-place	None
reverse()	Reverse in-place	None
copy()	Shallow copy	New list
sorted(lst)	Sorted copy	New list

DICTIONARY METHODS

Method	Action	Returns
get(key, def)	Safe value access	Value or default
keys()	All keys	dict_keys view
values()	All values	dict_values view
items()	All (key,val) pairs	dict_items view
update(other)	Merge dict	None (in-place)
pop(key)	Remove & get value	Value / KeyError
popitem()	Remove last item	(key,val) tuple
setdefault(k,v)	Get or set default	Value

<code>copy()</code>	Shallow copy	New dict
<code>clear()</code>	Remove all	None
<code>fromkeys(keys, v)</code>	Create from keys	New dict

SET METHODS

Method	Action	Returns
<code>add(x)</code>	Add x	None
<code>remove(x)</code>	Remove x (error if missing)	None/KeyError
<code>discard(x)</code>	Remove x (safe)	None
<code>pop()</code>	Remove arbitrary item	Removed item
<code>clear()</code>	Remove all	None
<code>union(t) / </code>	Items in s or t	New set
<code>intersection(t) / &</code>	Items in both	New set
<code>difference(t) / -</code>	Items in s not t	New set
<code>symmetric_diff(t) / ^</code>	Items in one only	New set
<code>issubset(t) / <=</code>	s all in t?	bool
<code>issuperset(t) / >=</code>	t all in s?	bool
<code>isdisjoint(t)</code>	No common items?	bool